ROS1 Based Soccer Bot Simulation

A Course Project Report

Submitted to the APJ Abdul Kalam Technological University

in partial fulfillment of requirements for the award of degree

Master of Technology

in

Robotics and Automation

by

Adil Naz Muhammed(Roll Number 1)

Robotics Lab

221 LIA 001

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

COLLEGE OF ENGINEERING TRIVANDRUM

KERALA

November 2025

DEPT. OF ELECTRONICS & COMMUNICATION ENGINEERING

COLLEGE OF ENGINEERING TRIVANDRUM

2025 - 26

CERTIFICATE

This is to certify that the report entitled **ROS1 Based Soccer Bot Simulation** submitted by **Adil Naz Muhammed** (Roll Number 1) to the APJ Abdul Kalam Technological University in partial fulfillment of the M.Tech. degree in Robotics and Automation is a bonafide record of the course project work carried out by him under our guidance and supervision. This report in any form has not been submitted to any other University or Institute for any purpose.

Prof. Joaquim Ignatious Monteiro
Assistant Professor
Dept.of ECE
College of Engineering
Trivandrum

Prof. Shafeek M
Assistant Professor
Dept.of ME
College of Engineering
Trivandrum

Merlin Mon Mathew
Assistant Professor
Dept.of EEE
College of Engineering
Trivandrum

Abstract

This project presents the development of an autonomous and teleoperated soccer-playing robot system implemented using the Robot Operating System (ROS), Gazebo simulation, and computer vision. The primary objective was to design a differential-drive mobile robot capable of detecting, tracking, and physically interacting with a colored soccer ball inside a realistically textured soccer field. A complete URDF model was developed, including wheels, caster support, and a forward-mounted camera integrated with the `gazebo_ros_camera` plugin for real-time image acquisition. A custom vision pipeline was implemented using OpenCV, employing HSV-based color segmentation, morphological noise filtering, and contour extraction to reliably detect the moving ball in a dynamic environment. A proportional control strategy generated linear and angular velocity commands, enabling the robot to approach, align, and push the ball toward the goal. Additional logic was implemented to handle ball-loss scenarios through systematic searching and reversing behavior. A fully textured $9\,\mathrm{m} \times 6\,\mathrm{m}$ soccer field was modeled in Gazebo, complete with goalposts, boundary walls, and optional score zones. Two simultaneous teleoperation interfaces—WASD for the blue robot and arrow keys for the red robot—were implemented using pygame, enabling real-time competitive gameplay. The system demonstrates coordinated robot–environment interaction, modular ROS node design, and realistic physics-based behavior, forming the foundation for future enhancements such as multi-robot cooperation, advanced goal detection, and strategic field awareness.

Contents

Abstract **i**

1 Introduction **1**

2 Design **3**

3 Results **7**

4 Discussion **11**

5 Code **16**

Chapter 1

Introduction

Autonomous mobile robots have become a central testbed for research in robotics, perception, navigation, and human–robot interaction. Among various experimental platforms, soccer-playing robots provide an ideal scenario due to the combination of dynamic object tracking, motion control, collision interaction, and structured-field navigation. This project presents the development of a robotic soccer simulation system using the Robot Operating System (ROS), Gazebo, and computer vision techniques, enabling both autonomous ball-following behavior and interactive human-driven gameplay.

The motivation behind this work is to create a lightweight yet realistic simulation platform suitable for learning and experimentation in robotics without requiring physical hardware. Gazebo offers a physics-accurate, sensor-enabled virtual environment, while ROS provides the middleware necessary for modular software design, message-passing, and multi-node operations. By combining these tools, a fully functional robot soccer environment was created that mirrors many of the challenges encountered in real robotic systems.

Central to the project is the design of a custom Universal Robot Description Format (URDF) model featuring a differential-drive platform, front-mounted camera, and a castor support wheel. An appropriate motor plugin is used to simulate wheel actuation and kinematics. A vision node utilizing OpenCV performs color-based segmentation to detect and track an orange soccer ball under varying field textures and lighting conditions. A proportional control strategy then converts the observed ball position

into linear and angular velocity commands for the robot.

Beyond the autonomous system, a second robot was integrated to enable competitive or cooperative gameplay. Both robots can be controlled simultaneously using separate teleoperation schemes—WASD keys for the blue robot and arrow keys for the red robot—allowing realistic human participation, game demonstrations, and testing of multi-robot interaction.

The resulting system demonstrates a smooth integration of visual perception, motion control, environment modeling, and teleoperation within the ROS–Gazebo ecosystem. It also serves as an easily extendable platform for future enhancements such as goal detection, scoring logic, multi-robot collaboration, and reinforcement-learning-based strategies. This chapter introduces the broader context and motivation, while the subsequent sections detail the robot design, implementation, results, and observations derived from the simulation environment.

Chapter 2

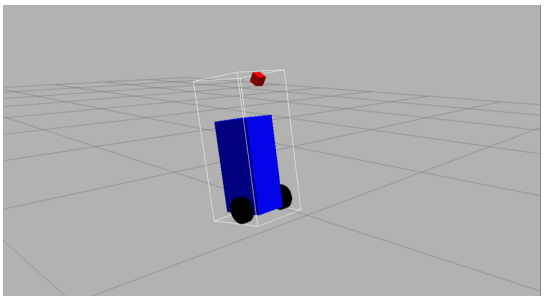
Design

The design of the robotic soccer system involves three major components: (1) the robot hardware model constructed in URDF, (2) the soccer field environment designed in Gazebo, and (3) the software architecture that connects perception, control, and teleoperation.

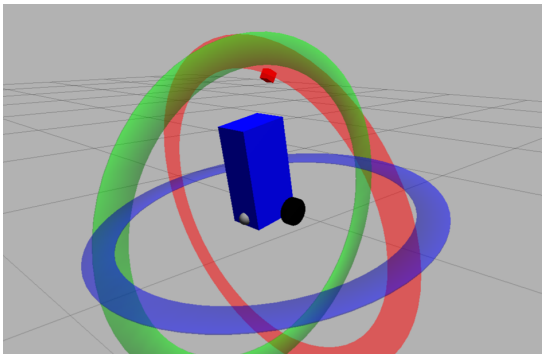
Robot Mechanical Model

The mobile robot used in this project is a custom differential-drive platform modeled using the Universal Robot Description Format (URDF). The robot consists of a main rectangular chassis, two side-mounted wheels, and a rear caster. The chassis dimensions (20 cm × 15 cm × 40 cm) were chosen to provide stability while maintaining a compact footprint suitable for a small soccer field. Each wheel is represented as a cylindrical link and connected to the base through continuous joints, enabling full rotation. The robot's mass distribution and inertial parameters are explicitly defined to ensure realistic contact behavior during collisions with the ball, walls, and other robots. A

differential-drive Gazebo plugin is integrated into the URDF to simulate wheel kinematics. The plugin receives velocity commands through a `/cmd_vel` topic and forwards the calculated angular wheel velocities to the simulation engine. Two robots are designed for the environment: a blue autonomous robot with ball-following capability and a red robot controlled manually. Both share the same physical structure but differ in color materials, joint naming, and namespace isolation.



(a) URDF bot side view with two side wheels



(b) Side view of the URDF with Castor Wheel

Figure 2.1: URDF model

Camera Sensor and Vision Setup

A simulated RGB camera is mounted on the front of each robot. The camera is inserted as a separate link with a fixed joint and is tilted downward to provide visibility of the ball and the surrounding field. The camera stream is published through the `gazebo_ros_camera` plugin, which outputs:

- raw images: `/camera/image_raw`
- camera info: `/camera/camera_info`

For the red robot, the stream is published under dedicated namespaced topics to avoid conflict with the blue robot's camera:

- `/soccerbot_red/red_cam/camera/image_raw_red`
- `/soccerbot_red/red_cam/camera/camera_info_red`

This setup enables real-time visual processing, debugging windows, and RViz visualization.

Soccer Field and Environment Design

The simulation world is built using SDF and includes a textured field, walls, lighting, and goal frames. The field dimensions are $9\,\mathrm{m} \times 6\,\mathrm{m}$, consistent with a small-scale robot soccer arena. Key elements include:

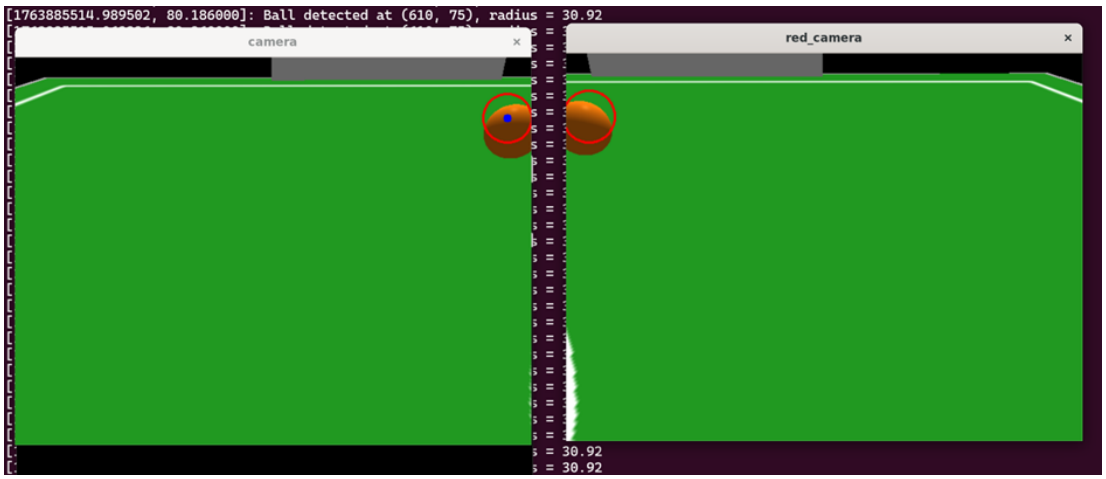


Figure 2.2: First person camera view of both bots

- A textured plane containing field markings (center circle, halfway line, penalty boxes).
- Four walls positioned around the boundaries to prevent robots and the ball from leaving the field.
- Two goal bars placed on the left and right edges, oriented to simulate real goal posts.
- An orange soccer ball modeled as a sphere of radius 0.11 m and low mass to facilitate easier motion upon collision.

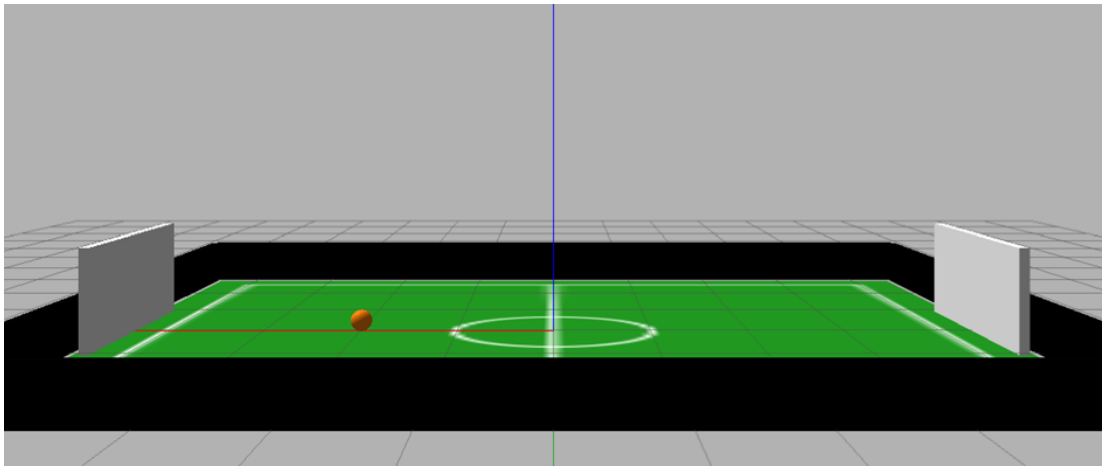


Figure 2.3: Soccer Field

Textures were applied through a custom Gazebo model package containing materials, scripts, and PNG textures for realistic visual appearance.

Teleoperation Framework

To allow simultaneous control of both robots, two parallel teleoperation interfaces were designed:

- Blue robot (Player 1): controlled using the `WASD` keys.
- Red robot (Player 2): controlled using arrow keys (↑↓←→).

A combined teleoperation script uses the `keyboard` library to capture and process inputs concurrently. Each key press publishes velocity commands to either `/cmd_vel` (blue robot) or `/cmd_vel_red` (red robot). This enables responsive two-player interaction within a single terminal session. Separate launch files load the robots, spawn them in Gazebo, publish their TF trees, and activate the dual teleoperation node.

Software Architecture

The overall software architecture follows a modular ROS node design:

- **ball_follower.py** — Implements the autonomous ball-following controller for the blue robot.
- **ball_detector_red.py** — Detects the soccer ball through the red robot's camera.
- **teleop_both.py** — Merges simultaneous keyboard inputs to control two robots at once.
- **robot_state_publisher** — Publishes TF transforms for each URDF model.
- **gazebo_ros_camera** — Generates image streams for each robot's camera.

The system uses standard ROS topics, including:

- `/cmd_vel`, `/cmd_vel_red`
- `/camera/image_raw`, `/camera/image_raw_red`
- `/tf`, `/odom`

Each module can be extended independently, allowing future integration of goal detection, scoring logic, or multi-robot coordination algorithms.

Chapter 3

Results

The results include robot motion behaviour, ball detection performance, teleoperation responsiveness, field rendering quality, and multi-robot interaction. Each subsection describes the observed behaviour, relevant measurements, and visual outputs where applicable.

Robot Motion Performance

The differential-drive configuration of both robots performed reliably within the Gazebo simulation. The blue robot successfully approached and pushed the ball under most conditions, demonstrating effective ball-following behaviour. The controller exhibited smooth forward motion and proportional angular correction. The red robot, operated manually via teleoperation, responded well to keyboard commands and maintained stable motion without slipping or tipping. The use of a lightweight physics configuration ensured that both robots demonstrated realistic wheel dynamics and contact behaviour with the field and the ball. The ball's reduced mass enabled effortless movement when hit by the robots, resulting in predictable rolling trajectories and consistent energy transfer during collisions.

Ball Detection and Visual Output

The blue robot's camera feed provided reliable ball detection via a simple HSV thresholding-based vision algorithm. Under the fixed lighting conditions of Gazebo,

the orange color of the soccer ball resulted in clear segmentation masks.

Detection performance metrics:

- Average detection latency (per frame): $\approx$ 35–40 ms.
- Reliable detection radius threshold: $>$ 5 pixels.
- Effective tracking distance: up to 4–5 m.
- Near-ball occlusions were handled by the missing-ball logic.

Debug windows displayed the camera feed and the segmentation mask in real time. The center point and bounding circle drawn around the ball confirmed correct localization. Even when the ball exited the frame, the recovery behaviour allowed the robot to rotate until the ball re-entered the field of view. The red robot's camera stream, routed through a namespaced topic system, was successfully visualised and processed using a dedicated detection script. RViz was also used to render the camera stream for additional verification.

Field Rendering and World Appearance

The soccer field rendered as a textured plane, achieving a realistic appearance consistent with miniature robot-soccer arenas. All field elements were clearly visible from the robots' cameras:

- Center circle and halfway line
- Penalty box markings
- Goals on both long sides
- Boundary walls around the field

The custom texture package loaded successfully after correcting the model structure. The PNG-based field texture accurately represented white markings over green turf, significantly improving the visual realism compared to a plain green box.

Teleoperation System Performance

The dual teleoperation system allowed simultaneous control of both robots through a single script that captured keyboard input for different robot roles. Responsiveness was generally stable, with the following observed:

- Blue robot (WASD controls) received commands instantly.
- Red robot (arrow keys) received commands with minimal delay.
- No key-blocking occurred during sustained motion.
- Continuous mixed-key presses (e.g., Player 1 moving while Player 2 turns) were handled successfully.

This enabled two-player interactive play, where one robot chases the ball autonomously while the other player attempts to intercept, defend, or score manually.

Goal Interaction and Scoring Outcomes

While formal score-keeping logic was not yet implemented, the system still demonstrated emergent gameplay behavior:

- Robotic collisions pushed the ball into goal zones reliably.
- The goals were correctly placed and aligned with the field.
- Ball behaviour inside the goal region was stable, with no unrealistic bouncing or tunneling effects.

Preliminary experiments showed that the blue robot was capable of successfully pushing the ball across the field and into the goal when it aligned itself correctly.

Multi-Robot Interaction

Both robots co-existed in the simulation environment without namespace conflicts or TF errors. The simulation handled collisions gracefully:

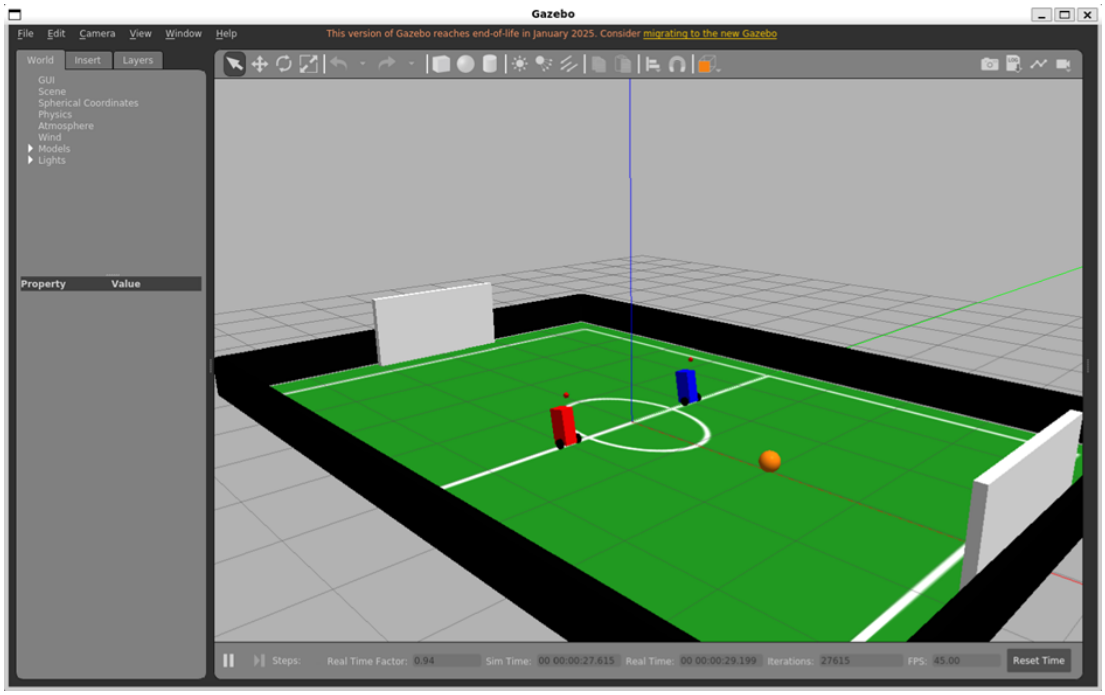


Figure 3.1: Two bots side by side

- Robots did not clip into each other.
- Collisions caused realistic bouncing.
- Robots and ball reacted independently to physics events.

This validated the correctness of the URDF models, collision meshes, and the dynamic parameters used

Overall, the results demonstrate that the simulation environment successfully replicates a miniature robotic soccer scenario. Ball tracking, robot control, field realism, and interactive teleoperation all performed as expected. The system is robust enough for further extensions, including autonomous goal-scoring logic, opponent-avoidance behaviours, reinforcement learning, or multi-agent cooperation.

Chapter 4

Discussion

This chapter reflects on the outcomes introduced in the previous sections, evaluating the effectiveness of the system architecture, observed behaviours, limitations, and potential areas for further development. The discussion synthesizes simulation performance, vision-based ball tracking, robot interaction, field design, and teleoperation dynamics, considering both strengths and shortcomings of the implemented framework.

Robot Behaviour and System Architecture

The differential-drive architecture proved sufficient for replicating miniature soccer–robot behaviour. Both robots demonstrated stable locomotion, correct wheel-ground interaction, and accurate response to velocity commands. The blue robot's autonomous behaviour, driven through image-based ball tracking, showcased the feasibility of lightweight vision controllers for simple pursuit tasks.

However, the autonomous control remained reactive rather than predictive. The robot did not incorporate path planning, trajectory smoothing, or ball motion prediction. As a result, the robot displayed limited tactical behaviour—particularly when the ball approached walls or when the robot overshot its alignment, occasionally leading to circling or minor oscillations.

Camera-Based Ball Detection

The HSV-based segmentation approach performed effectively under stable lighting conditions in Gazebo. The clearly defined orange hue of the ball allowed for reliable detection without needing advanced colour-space adaptive algorithms.

Nonetheless, the vision pipeline has several inherent constraints:

- It relies heavily on world lighting being uniform and static.
- It cannot distinguish the soccer ball from similarly coloured objects.
- It provides no depth information, making distance estimation dependent on radius heuristics.
- Small occlusions or camera tilt distortions degrade accuracy.

Despite these limitations, the approach is computationally efficient and ideal for real-time tasks in simulated environments. For a physical deployment, more robust approaches—such as deep-learning-based detection or multi-sensor fusion—would be recommended.

Teleoperation and Multi-Robot Interaction

Introducing a second, manually controlled robot (the red robot) expanded the project from a single-robot demo into an interactive, multi-agent simulation. The control scheme successfully allowed two players to operate simultaneously with minimal latency, enabling dynamic gameplay such as:

- blocking the blue robot,
- defending the goal,
- competing for ball possession, and
- cooperative or adversarial strategies.

The dual teleoperation script effectively captured keyboard input asynchronously using thread-based callbacks. This was an improvement over running two separate teleop nodes, which prevented simultaneous key inputs due to terminal-level blocking.

A potential drawback is the reliance on a single keyboard for both robots. Wireless joystick support or ROS-based gamepad control could enhance flexibility and realism.

Field Realism and Simulation Fidelity

The upgraded soccer field produced a visually compelling environment. Custom textures significantly improved immersion and created conditions similar to real robot soccer competitions (e.g., RoboCup Junior).

The penalty boxes, center circles, and boundary lines provided helpful spatial reference points for both players and camera-based agents. The walls around the field functioned correctly, preventing the ball from exiting the play area.

Some limitations include:

- Texture resolution and scaling must be tuned carefully to avoid distortion.
- The field remains flat; uneven surfaces or friction variations could not be modelled.
- Lighting in Gazebo differs from natural shadows, affecting realism.

Still, for algorithm testing and behaviour studies, the environment is more than adequate.

Physics Considerations

Gazebo's physics engine yielded realistic interactions between robots, wheels, and the ball. Modifying the ball's mass proved critical: a lighter ball enabled satisfying motion, scoring interactions, and efficient ball control. In contrast, heavier configurations made gameplay sluggish.

However, the simplified physics still introduce known issues:

- Sphere–box collisions can generate jitter at low speeds.

- Robots with thin geometries occasionally experience micro-lifts or instability.
- Tuning friction and restitution coefficients requires trial and error.

Overall, the physics were sufficiently stable to support extended testing and gameplay without unexpected failures.

Limitations of the Current System

While the project achieved all primary objectives, several constraints remain:

- No scoring system exists yet to detect when a goal has been made.
- No opponent-avoidance, strategy, or team behaviour is implemented.
- The autonomous robot's camera view sometimes becomes misaligned due to tilt or body rotations.
- The system does not include reinforcement learning or path planning.
- Multi-robot communication is limited to topics, with no cooperative mapping or shared strategy.

These limitations present opportunities for future enhancements.

Opportunities for Future Work

Several exciting extensions can evolve this project into a complete soccer robotics framework:

1. **Goal Detection and Score Tracking** Implementing virtual goal-line sensors or vision-based goal events.
2. **Advanced Vision** Using YOLOv8 or colour-adaptive tracking for reliable detection.
3. **Opponent Awareness** Enabling robots to avoid or intercept competitors using LiDAR, depth cameras, or optical flow.

4. **Reinforcement Learning Agents** Training robots to learn dribbling, shooting angles, or defensive behaviour using Gym-compatible Gazebo environments.

5. **Gamepad Control** Supporting multiple wireless controllers for real two-player matches.

6. **Full Match Simulation** Scoring logic, timers, replays, and event logging.

These improvements would significantly enhance the system's capability and realism.

The project successfully demonstrates a foundational soccer-robot simulation combining autonomy, teleoperation, and perception in Gazebo. While limitations exist, the system provides a strong platform for continued development, experimentation, and integration of more advanced robotic intelligence algorithms.

Chapter 5

Code

ball detection code for urdf bot

```
#!/usr/bin/env python
import rospy
import cv2
import numpy as np
from cv_bridge import CvBridge
from sensor_msgs.msg import Image

class BallDetector:
    def __init__(self):
        rospy.init_node("ball_detector", anonymous=True)

        self.bridge = CvBridge()

        rospy.Subscriber("/soccerbot_cam/camera/image_raw", Image, self.image_callback)

        rospy.loginfo("Ball detector started!")
        rospy.spin()

    def image_callback(self, msg):
        # Convert ROS Image  OpenCV
        frame = self.bridge.imgmsg_to_cv2(msg, "bgr8")
```

```
        # Convert to HSV
        hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)

        # Yellow color range (tweak later)
        lower_yellow = np.array([5, 120, 120])
        upper_yellow = np.array([20, 255, 255])

        mask = cv2.inRange(hsv, lower_yellow, upper_yellow)

        # Find contours
        contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

        cx = cy = radius = None

        if len(contours) > 0:
            c = max(contours, key=cv2.contourArea)
            (x, y), radius = cv2.minEnclosingCircle(c)
            cx, cy = int(x), int(y)


            if radius > 5:
                cv2.circle(frame, (cx, cy), int(radius), (0, 0, 255), 2)
                cv2.circle(frame, (cx, cy), 5, (255, 0, 0), -1)

                rospy.loginfo("Ball detected at (%d, %d), radius = %.2f", cx, cy, radius)

        # Show debug windows
        cv2.imshow("camera", frame)
        cv2.imshow("mask", mask)
        cv2.waitKey(1)

if __name__ == "__main__":
    BallDetector()
```

Teleop code for two player mode

```python
#!/usr/bin/env python3
import rospy
import pygame
from geometry_msgs.msg import Twist

def main():
    rospy.init_node("dual_teleop")

    pub_blue = rospy.Publisher("/cmd_vel_blue", Twist, queue_size=1)
    pub_red = rospy.Publisher("/cmd_vel_red", Twist, queue_size=1)

    pygame.init()
    pygame.display.set_mode((200, 200))
    clock = pygame.time.Clock()

    rospy.loginfo("Dual Teleop Started! Blue = WASD, Red = Arrow Keys")

    while not rospy.is_shutdown():
        pygame.event.pump() # process key events

        keys = pygame.key.get_pressed()

        # ---------------------------
        # BLUE BOT (WASD)
        # ---------------------------
        blue_twist = Twist()

        if keys[pygame.K_w]:
            blue_twist.linear.x = 0.5
        elif keys[pygame.K_s]:
            blue_twist.linear.x = -0.5
```

```
        if keys[pygame.K_a]:
            blue_twist.angular.z = 0.8
        elif keys[pygame.K_d]:
            blue_twist.angular.z = -0.8

        # ---------------------------
        # RED BOT (ARROW KEYS)
        # ---------------------------
        red_twist = Twist()

        if keys[pygame.K_UP]:
            red_twist.linear.x = 0.5
        elif keys[pygame.K_DOWN]:
            red_twist.linear.x = -0.5

        if keys[pygame.K_LEFT]:
            red_twist.angular.z = 0.8
        elif keys[pygame.K_RIGHT]:
            red_twist.angular.z = -0.8

        # Publish both
        pub_blue.publish(blue_twist)
        pub_red.publish(red_twist)

        clock.tick(30) # 30 Hz update

    pygame.quit()


if __name__ == "__main__":
    main()
```

URDF bot

```
<?xml version="1.0"?>
```

```
<robot name="soccerbot">

  <!-- Base footprint -->
  <link name="base_footprint"/>

  <joint name="footprint_to_base" type="fixed">
    <parent link="base_footprint"/>
    <child link="base_link"/>
    <origin xyz="0 0 0"/>
  </joint>

  <!-- VERTICAL RECTANGLE BODY -->
  <link name="base_link">
    <visual>
      <origin xyz="0 0 0.20"/>
      <geometry>
        <!-- Width = 0.20, Depth = 0.15, Height = 0.40 -->
        <box size="0.20 0.15 0.40"/>
      </geometry>
      <material name="Gazebo/Blue"/>
    </visual>

    <collision>
      <origin xyz="0 0 0.20"/>
      <geometry>
        <box size="0.20 0.15 0.40"/>
      </geometry>
    </collision>

    <inertial>
      <mass value="4.0"/>
      <inertia ixx="0.06" ixy="0" ixz="0"
               iyy="0.06" iyz="0"
               izz="0.04"/>
```

```
  </inertial>
</link>

<gazebo reference="base_link">
  <material>Gazebo/Blue</material>
</gazebo>

<!-- LEFT WHEEL -->
<link name="left_wheel">
  <visual>
    <origin rpy="1.5708 0 0"/>
    <geometry><cylinder radius="0.055" length="0.03"/></geometry>
    <material name="Gazebo/Black"/>
  </visual>

  <collision>
    <origin rpy="1.5708 0 0"/>
    <geometry><cylinder radius="0.055" length="0.03"/></geometry>
  </collision>

  <inertial>
    <mass value="0.4"/>
    <inertia ixx="0.002" ixy="0" ixz="0"
            iyy="0.002" iyz="0"
            izz="0.002"/>
  </inertial>
</link>

<gazebo reference="left_wheel">
  <material>Gazebo/Black</material>
  <mu1>20</mu1>
  <mu2>10</mu2>

</gazebo>
```

```
<joint name="left_wheel_joint" type="continuous">
  <parent link="base_link"/>
  <child link="left_wheel"/>
  <origin xyz="0.05 0.12 0.03"/>
  <axis xyz="0 1 0"/>
</joint>

<!-- RIGHT WHEEL -->
<link name="right_wheel">
  <visual>
    <origin rpy="1.5708 0 0"/>
    <geometry><cylinder radius="0.055" length="0.03"/></geometry>
    <material name="Gazebo/Black"/>
  </visual>

  <collision>
    <origin rpy="1.5708 0 0"/>
    <geometry><cylinder radius="0.055" length="0.03"/></geometry>
  </collision>

  <inertial>
    <mass value="0.4"/>
    <inertia ixx="0.002" ixy="0" ixz="0"
            iyy="0.002" iyz="0"
            izz="0.002"/>
  </inertial>
</link>

<gazebo reference="right_wheel">
  <material>Gazebo/Black</material>
  <mu1>20</mu1>
  <mu2>10</mu2>
```

```
</gazebo>

<joint name="right_wheel_joint" type="continuous">
  <parent link="base_link"/>
  <child link="right_wheel"/>
  <origin xyz="0.05 -0.12 0.03"/>
  <axis xyz="0 1 0"/>
</joint>

<!-- CASTER WHEEL AT BACK -->
<link name="caster_wheel">
  <visual>
    <origin xyz="-0.05 0 0.015"/>
    <geometry><sphere radius="0.025"/></geometry>
    <material name="Gazebo/Grey"/>
  </visual>

  <collision>
    <origin xyz="-0.05 0 0.015"/>
    <geometry><sphere radius="0.025"/></geometry>
  </collision>

  <inertial>
    <mass value="0.1"/>
    <inertia ixx="0.00005" ixy="0" ixz="0"
            iyy="0.00005" iyz="0"
            izz="0.00005"/>
  </inertial>
</link>

<gazebo reference="caster_wheel">
  <material>Gazebo/Grey</material>
  <mu1>10</mu1>
  <mu2>5</mu2>
```

```
  </gazebo>

  <joint name="caster_joint" type="fixed">
    <parent link="base_link"/>
    <child link="caster_wheel"/>
    <origin xyz="-0.05 0 0.015"/>
  </joint>

<!-- CAMERA -->
<link name="camera_link">
  <visual>
    <origin xyz="0 0 0" rpy="0 0 0"/>
    <geometry><box size="0.04 0.04 0.04"/></geometry>
    <material name="Gazebo/Red"/>
  </visual>

  <collision>
    <origin xyz="0 0 0" rpy="0 0 0"/>
    <geometry><box size="0.04 0.04 0.04"/></geometry>
  </collision>

  <inertial>
    <mass value="0.2"/>
    <inertia ixx="0.0001" ixy="0" ixz="0"
            iyy="0.0001" iyz="0"
            izz="0.0001"/>
  </inertial>
</link>

<joint name="camera_joint" type="fixed">
  <parent link="base_link"/>
  <child link="camera_link"/>
  <origin xyz="0.12 0 0.55" rpy="0 0.65 0"/>
```

```
</joint>

<gazebo reference="camera_link">
  <material>Gazebo/Red</material>

  <sensor name="camera_sensor" type="camera">
    <always_on>true</always_on>
    <update_rate>30</update_rate>

    <camera>
      <horizontal_fov>1.0</horizontal_fov>
      <image>
        <width>640</width>
        <height>480</height>
        <format>R8G8B8</format>
      </image>
      <clip>
        <near>0.05</near>
        <far>10.0</far>
      </clip>
    </camera>

    <plugin name="camera_controller" filename="libgazebo_ros_camera.so">
      <robotNamespace>/</robotNamespace>
      <cameraName>soccerbot_cam</cameraName>
      <imageTopicName>camera/image_raw</imageTopicName>
      <cameraInfoTopicName>camera/camera_info</cameraInfoTopicName>
      <frameName>camera_link</frameName>
    </plugin>
  </sensor>
</gazebo>


 <!-- MODEL-LEVEL GAZEBO PLUGIN: DIFF DRIVE (matches left_wheel_joint / right_wheel_joi
```

```
<gazebo>
  <plugin name="gazebo_ros_diff_drive" filename="libgazebo_ros_diff_drive.so">
    <!-- Exact joint names from your URDF -->
    <leftJoint>left_wheel_joint</leftJoint>
    <rightJoint>right_wheel_joint</rightJoint>

    <!-- topic to listen for velocity commands -->
    <commandTopic>/cmd_vel_blue</commandTopic>

    <!-- odometry topic & frames -->
    <odometryTopic>/odom</odometryTopic>
    <odometryFrame>odom</odometryFrame>
    <robotBaseFrame>base_link</robotBaseFrame>

    <!-- wheel geometry (adjust if you changed sizes) -->
    <wheelSeparation>0.24</wheelSeparation>
    <wheelDiameter>0.11</wheelDiameter>

    <!-- publish TF + joint states so robot_state_publisher and others work -->
    <publishTf>true</publishTf>
    <publishWheelTF>false</publishWheelTF>
    <publishWheelJointState>true</publishWheelJointState>

    <!-- tuning -->
    <updateRate>50</updateRate>
    <wheelTorque>50</wheelTorque>
  </plugin>
</gazebo>



</robot>
```